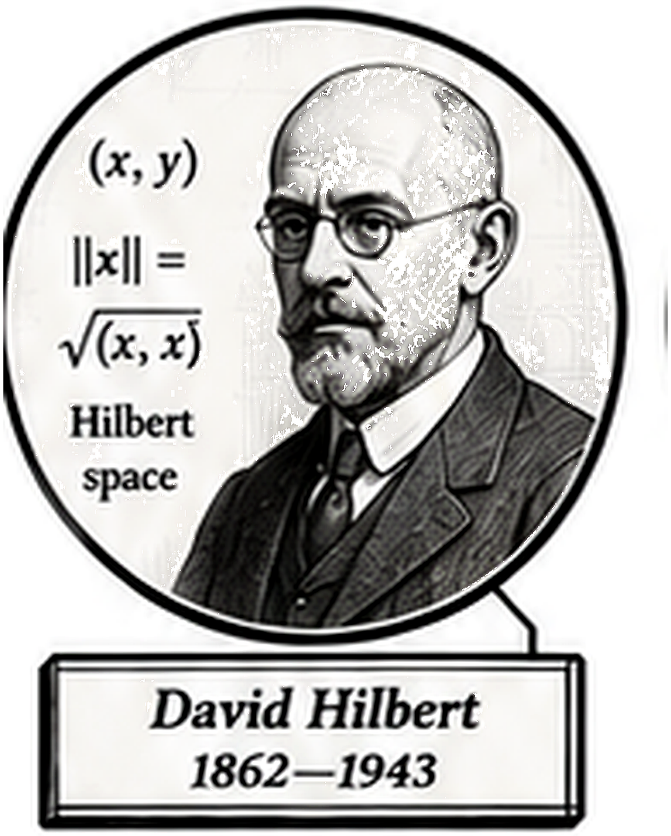
Volume VII

Advanced Logic



Frontispiece placeholder.
David Hilbert, 1862–1943

Volume VII

Advanced Logic

Self-Study Notes

William Sollers

June 26, 2026

Contents

Part I

Advanced Logic

Chapter 1

Category Theory

category-theory

Numerical PDE $\to$ **Category Theory** $\to$ Model Theory

1.1 Category Theory

Remark (Planned content). Active mathematical content has not yet been added.

Proofs

Capstone Problem

Problem. TODO: state one synthesizing problem for Category Theory, using only concepts at or before this chapter in the registry.

Remark (Dependency ceiling). The capstone may use only results routed at or before this chapter.

Capstone

Chapter 2

Model Theory

model-theory

Category Theory → **Model Theory** → Proof Theory

2.1 Languages and Structures

L-Structures Toolkit — Quick Reference

Concept	**Symbol / Notation**	**Detail**
First-order language	$\mathcal{L}$	↓
L-structure	$\mathcal{M} = (M, \mathcal{L}^{\mathcal{M}})$	↓
Domain / universe	$\lvert\mathcal{M}\rvert$ or M	↓
Interpretation of f	$f^{\mathcal{M}} : M^n \to M$	↓
Interpretation of R	$R^{\mathcal{M}} \subseteq M^n$	↓
Interpretation of c	$c^{\mathcal{M}} \in M$	↓
Variable assignment	$s : \mathrm{Var} \to M$	↓
Term evaluation	$s(t) \in M$	↓
Satisfaction	$\mathcal{M} \models \varphi[s]$	↓

First-Order Languages

Definition (First-Order Language $\mathcal{L}$)

Definition 2.1.1 (First-Order Language $\mathcal{L}$)**.** A **first-order language** $\mathcal{L}$ consists of three (possibly empty) sets of non-logical symbols:

- a set $\mathcal{R}$ of **relation symbols** (also called *predicate symbols*), each assigned a fixed arity $n \geq 1$;
- a set $\mathcal{F}$ of **function symbols**, each assigned a fixed **arity** $n \geq 0$ (0-ary function symbols are *constant symbols*);
- a set $\mathcal{C}$ of **constant symbols** (equivalently, 0-ary function symbols — listed separately for emphasis).

Together with the logical symbols common to all first-order languages (variables, connectives $\neg, \wedge, \vee, \to, \leftrightarrow$, quantifiers $\forall, \exists$, equality $=$, parentheses), the triple $(\mathcal{F}, \mathcal{R}, \mathcal{C})$ determines $\mathcal{L}$.

Remark (Dependencies).

- Arity
- Relation Symbol
- Function Symbol
- Constant Symbol
- Variable
- Logical Connective

Remark (Fully quantified form). $\mathcal{L} = (\mathcal{F}, \mathcal{R}, \mathcal{C})$ where $\mathrm{ar}(f) \in \mathbb{N}$ for each $f \in \mathcal{F}$ and $\mathrm{ar}(R) \in \mathbb{N}_{\geq 1}$ for each $R \in \mathcal{R}$.

Remark (Intuition). A language is a *vocabulary*: it names the operations, relations, and distinguished elements that can appear in sentences, without yet saying what those names mean. The meaning is supplied by a structure.

Remark (Source note). Some sources (e.g. Marker §1.1, Hodges) absorb constant symbols into 0-ary function symbols. Others (e.g. Chang–Keisler) list them separately. The content is equivalent; only the bookkeeping differs.

Remark (Consequence). The set of $\mathcal{L}$-terms and the set of $\mathcal{L}$-formulas are both defined inductively from $\mathcal{L}$; see term evaluation below.

Example (Standard examples of first-order languages).

1. **Language of ordered fields** $\mathcal{L}_{\mathrm{of}}$: function symbols $\{+, \cdot, -\}$ of arities $2, 2, 1$; relation symbol $\{<\}$ of arity 2; constant symbols $\{0, 1\}$. The real field $(\mathbb{R}, +, \cdot, -, <, 0, 1)$ is an $\mathcal{L}_{\mathrm{of}}$-structure.
2. **Language of graphs** $\mathcal{L}_{\mathrm{gr}}$: no function symbols, no constants; one binary relation symbol $\{E\}$ (for "edge").

3. **Language of groups** $\mathcal{L}_{\text{gp}}$: function symbols $\{\cdot, {}^{-1}\}$ of arities $2, 1$; constant symbol $\{e\}$.

4. **Pure equality language** $\mathcal{L}_{=}$: no non-logical symbols at all. Structures are arbitrary non-empty sets.

L-Structures

Definition ($\mathcal{L}$-Structure)

Definition 2.1.2 ($\mathcal{L}$-Structure)**.** Let $\mathcal{L} = (\mathcal{F}, \mathcal{R}, \mathcal{C})$ be a first-order language. An **$\mathcal{L}$-structure $\mathcal{M}$** consists of:

1. a non-empty set M called the **domain** (or **universe**) of $\mathcal{M}$, written $|\mathcal{M}|$;
2. for each n-ary relation symbol $R \in \mathcal{R}$, a relation $R^{\mathcal{M}} \subseteq M^n$;
3. for each n-ary function symbol $f \in \mathcal{F}$, a function $f^{\mathcal{M}} : M^n \to M$;
4. for each constant symbol $c \in \mathcal{C}$, a distinguished element $c^{\mathcal{M}} \in M$.

The superscript notation $f^{\mathcal{M}}, R^{\mathcal{M}}, c^{\mathcal{M}}$ denotes the **interpretation** of each symbol in $\mathcal{M}$.

Remark (Dependencies).

- First-Order Language
- Arity
- Relation Symbol
- Function Symbol
- Constant Symbol
- Relation
- Function
- Cartesian Product
- Subset

Remark (Fully quantified form). $\mathcal{M}$ is an $\mathcal{L}$-structure iff $M \neq \emptyset$, and $\forall f \in \mathcal{F}_{(n)},\ f^{\mathcal{M}} : M^n \to M$, and $\forall R \in \mathcal{R}_{(n)},\ R^{\mathcal{M}} \subseteq M^n$, and $\forall c \in \mathcal{C},\ c^{\mathcal{M}} \in M$.

Remark (Intuition). A structure is an $\mathcal{L}$-vocabulary *made concrete*: each symbol is assigned an actual mathematical object of the correct type. The domain provides the raw material; the interpretations give the symbols their meaning inside that domain.

Remark (Common error). The domain M must be *non-empty*. First-order logic with an empty domain is pathological: $\forall x\, \varphi$ would be vacuously true and $\exists x\, \varphi$ vacuously false for every φ, breaking the duality of the quantifiers.

Remark (Source note)*.* Some sources write $\mathfrak{M}$ (Fraktur) for structures and reserve M for the domain; others use $\mathcal{M}$ throughout and write $|\mathcal{M}|$ for the domain. The latter convention is used here. Marker uses $\mathcal{M}$; Enderton uses $\mathfrak{A}$.

Example (L-structures for familiar mathematical objects)*.*

1. $(\mathbb{R}, +^{\mathbb{R}}, \cdot^{\mathbb{R}}, -^{\mathbb{R}}, <^{\mathbb{R}}, 0^{\mathbb{R}}, 1^{\mathbb{R}})$ is an $\mathcal{L}_{\text{of}}$-structure with domain $M = \mathbb{R}$.

2. $(\mathbb{Q}, +^{\mathbb{Q}}, \cdot^{\mathbb{Q}}, -^{\mathbb{Q}}, <^{\mathbb{Q}}, 0^{\mathbb{Q}}, 1^{\mathbb{Q}})$ is another $\mathcal{L}_{\text{of}}$-structure with domain $M = \mathbb{Q}$.

3. The complete graph K_3 on $\{1, 2, 3\}$ is an $\mathcal{L}_{\text{gr}}$-structure with $E^{K_3} = \{(1, 2), (2, 1), (1, 3), (3, 1), (2, 3),$

Two different structures can share the same language — the language only fixes the *type* of the interpretations, not their specific content.

Variable Assignments

Definition (Variable Assignment)

Definition 2.1.3 (Variable Assignment)**.** Let $\mathcal{M}$ be an $\mathcal{L}$-structure with domain M, and let $\mathrm{Var} = \{v_1, v_2, v_3, \ldots\}$ be a fixed countable set of first-order variables.
A **variable assignment** (or **environment**) for $\mathcal{M}$ is a function $s : \mathrm{Var} \to M$.
For a variable assignment s, a variable v_i, and an element $a \in M$, write $s[v_i \mapsto a]$ for the assignment that agrees with s on all variables except v_i, where it returns a:

$$s[v_i \mapsto a](v_j) = \begin{cases} a & \text{if } j = i, \\ s(v_j) & \text{if } j \neq i. \end{cases}$$

Remark (Dependencies)*.*

- $\mathcal{L}$-Structure
- Variable
- Function
- Countable

Remark (Fully quantified form)*.* $s : \mathrm{Var} \to M$, and $s[v_i \mapsto a](v_j) = a$ if $j = i$, else $s(v_j)$.

Remark (Intuition)*.* A variable assignment gives temporary values to free variables before evaluating a formula. The $s[v_i \mapsto a]$ notation is the standard tool for handling quantifier rules: to check $\exists v_i\, \varphi$, test whether $\mathcal{M} \models \varphi[s[v_i \mapsto a]]$ for some $a \in M$.

Term Evaluation

Definition (Term Evaluation)

Definition 2.1.4 (Term Evaluation)**.** Let $\mathcal{M}$ be an $\mathcal{L}$-structure and $s : \mathrm{Var} \to M$ a variable assignment. The **evaluation** $s(t) \in M$ of an $\mathcal{L}$-term t under s is defined inductively:

1. If $t = v_i$ is a variable, then $s(t) = s(v_i)$.
2. If $t = c$ is a constant symbol, then $s(t) = c^{\mathcal{M}}$.
3. If $t = f(t_1, \ldots, t_n)$ for an n-ary function symbol f, then $s(t) = f^{\mathcal{M}}(s(t_1), \ldots, s(t_n))$.

Remark (Dependencies).

- $\mathcal{L}$-Structure
- Variable Assignment
- Term
- Variable
- Arity
- Function Symbol
- Constant Symbol

Remark (Fully quantified form)*.* $s : \mathrm{Var} \to M$, and $s(t) \in M$ for every $\mathcal{L}$-term t. Formally: $s(\cdot)$ is the unique function on terms satisfying clauses (1)–(3).

Remark (Intuition)*.* Term evaluation is the "computation" step: given concrete values for all variables, every term reduces to an element of M. The inductive definition mirrors the inductive definition of terms themselves.

Satisfaction [Stub]

Definition (Satisfaction, $\mathcal{M} \models \varphi[s]$) — Stub

Let $\mathcal{M}$ be an $\mathcal{L}$-structure, s a variable assignment, and φ an $\mathcal{L}$-formula. The **satisfaction relation** $\mathcal{M} \models \varphi[s]$ (read: "$\mathcal{M}$ satisfies φ under s") is defined inductively on the structure of φ:

[To be filled in — clauses for atomic formulas, negation, conjunction, disjunction, implication, existential quantifier, universal quantifier.]

Remark (Fully quantified form)*.* *[Stub — to be completed.]*

Remark (Intuition)*.* Satisfaction is the semantic notion of truth: $\mathcal{M} \models \varphi[s]$ means φ is true in $\mathcal{M}$ when each free variable v_i takes the value $s(v_i)$. The definition proceeds by induction on formula complexity, grounding out on atomic formulas evaluated via term evaluation.

Elementary Equivalence and Substructures [Stub]

Proofs

Capstone

Capstone Problem

Problem. TODO: state one synthesizing problem for Model Theory, using only concepts at or before this chapter in the registry.

Remark (Dependency ceiling). The capstone may use only results routed at or before this chapter.

Chapter 3

Proof Theory

proof-theory

Model Theory $\to$ **Proof Theory** $\to$ Type Theory

3.1 Proof Theory

Remark (Planned content). Active mathematical content has not yet been added.

Proofs

Capstone Problem

Problem. TODO: state one synthesizing problem for Proof Theory, using only concepts at or before this chapter in the registry.

Remark (Dependency ceiling). The capstone may use only results routed at or before this chapter.

Capstone

Chapter 4

Type Theory

type-theory

Proof Theory $\rightarrow$ **Type Theory** $\rightarrow$ Lambda Calculus

4.1 Type Theory

Remark (Planned content). Active mathematical content has not yet been added.

Proofs

Capstone Problem

Problem. TODO: state one synthesizing problem for Type Theory, using only concepts at or before this chapter in the registry.

Remark (Dependency ceiling). The capstone may use only results routed at or before this chapter.

Capstone

Chapter 5

Lambda Calculus

lambda-calculus

Type Theory $\to$ **Lambda Calculus** $\to$ Algebras of Sets

5.1 Lambda Terms

Section Toolkit — Quick Reference

Concept	One-line summary
λ-term	Variable x, abstraction $\lambda x.M$, application MN
α-equivalence	$\lambda x.M =_\alpha \lambda y.M[y/x]$ when y fresh
β-reduction	$(\lambda x.M)N \to_\beta M[N/x]$
Normal form	Term with no β-redex; may not exist (non-termination)
Church boolean **T**	$\lambda x.\lambda y.x$
Church boolean **F**	$\lambda x.\lambda y.y$
Church numeral $\overline{n}$	$\lambda f.\lambda x.f^n(x)$ (f applied n times to x)
Type $\tau \to \sigma$	Function type: input of type τ, output of type σ
Typing judgment	$\Gamma \vdash M : \tau$ reads "M has type τ in context Γ"
Curry–Howard	Propositions are types; proofs are terms; $A \to B$ is implication

Untyped Lambda Calculus

Background References

- **First-order logic:** Enderton [1972], Barwise [1977a].
- **General topology:** Kelly [1955].
- **Set theory:** Halmos [1960], van Dalen et al. [1978].
- **Recursion theory:** Rogers [1967].
- **Category theory:** Arbib and Manes [1975], MacLane [1972].

Definition — Lambda Terms

Definition 5.1.1 (Lambda Terms)**.** The set Λ of λ-terms is defined inductively as follows:

1. Every variable x is a λ-term.
2. If M is a λ-term and x is a variable, then
$$\lambda x.\, M$$
is a λ-term.
3. If M and N are λ-terms, then
$$MN$$
is a λ-term.

Remark (Standard quantified statement)*.*

$$M \in \Lambda \quad \Longleftrightarrow \quad M \text{ is generated from variables by finitely many applications of } \lambda x.(\,\cdot\,) \text{ and } (\,\cdot\,)(\,\cdot\,).$$

Remark (Interpretation)*.* The λ-terms are the smallest set closed under three constructions: variables, abstraction (function formation), and application (function evaluation). This inductive definition plays the same role as recursive definitions in analysis or algebra: every λ-term is built from variables by finitely many applications of abstraction and application.

Remark (Structural View)*.* Every λ-term has a tree structure:

- variables are leaves,
- abstractions $\lambda x.\, M$ are unary nodes,
- applications MN are binary nodes.

Thus Λ is a freely generated syntactic algebra.

Proofs

Capstone

Capstone Problem

Problem. TODO: state one synthesizing problem for Lambda Calculus, using only concepts at or before this chapter in the registry.

Remark (Dependency ceiling). The capstone may use only results routed at or before this chapter.

Bibliography